

5 Implementation and Testing

This chapter details the physical realisation and quality assurance of the system, documenting the transition from the architectural models established in Chapter 4 into a functional software application. The chapter is divided into two primary sections to distinctly separate the development and validation phases.

Section 5.1 focuses on the implementation of the system. It outlines the specific environment configurations, including the server setup, database schema instantiation, and security parameters, required to deploy the 2-tier client-server architecture. Furthermore, this section highlights critical code snippets to demonstrate how core system requirements and advanced design patterns (such as the Singleton temporal engine and Proxy data masking) were practically translated into the PHP and MySQL backend.

Section 5.2 presents the testing phase, detailing the rigorous validation processes undertaken to ensure system reliability, data integrity, and strict requirement fulfillment. The testing procedures are comprehensively documented through structured, tabulated test cases that cover the core modules, validating both expected behaviors and boundary conditions to ensure the overall stability of the deployed solution.

IMPORTANT NOTE TO STUDENTS: Include details about:

- **Implementation**

A detailed description of how you actually carried out the implementation (e.g., coding, etc.) of your system/prototype.

- Include code snippets and descriptions to show how the requirements of the application/prototype have been met –
 - o For Smart Campus projects, code snippets of the MQTT protocol for both client side and server side has to be included with explanation.
- Include descriptions of important settings - Setting for server, network protocol, IP, database, security
- Note: this should **not** be a chronological account of the work you carried out.

- **Testing**

All test cases that have been carried out must be provided - To tabulate the test cases in tables

Note: Students may also opt to split Implementation and Testing into 2 separate chapters.

5.1 Implementation

This section details the physical realisation of the Supervisor Selection and Allocation System (SSAS), translating the architectural models and data schemas established in Chapter 4 into a functional, secure web application. In strict accordance with the project requirements, this section avoids a chronological development log, instead, it provides a structured, technical demonstration of how the system's Functional Requirements (FR) and Non-Functional Requirements (NFR) were fulfilled, database connection parameters, and exact code implementations utilised to engineer the core design patterns and backend business logic.

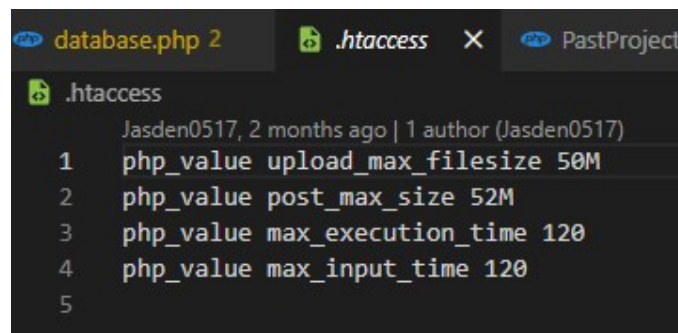
5.1.1 Development Environment and System Configurations

To fulfill the zero-cost budget constraint and support rapid prototyping, the system was developed and hosted locally utilising the XAMPP stack. The underlying configurations were strictly tailored to enforce the security and capacity requirements of the student-facing modules.

Web Server and Network Infrastructure

The application is hosted on an Apache web server bound to the local loopback address (localhost / 127.0.0.1) operating over standard HTTP port 80 during development. To manage the high volume of media and PDF proposal submissions without triggering server-level HTTP 413 (Payload Too large) errors, the Apache .htaccess file was explicitly configured to establish high outer ceilings for data transmission:

- `php_value upload_max_filesize 50M`
- `php_value post_max_size 52M`

A screenshot of a code editor showing the configuration of an Apache .htaccess file. The editor has a dark theme. The top of the editor shows three tabs: 'database.php 2', '.htaccess', and 'PastProject'. The active tab is '.htaccess'. The code in the .htaccess file is as follows:

```
.htaccess
Jasden0517, 2 months ago | 1 author (Jasden0517)
1  php_value upload_max_filesize 50M
2  php_value post_max_size 52M
3  php_value max_execution_time 120
4  php_value max_input_time 120
5
```

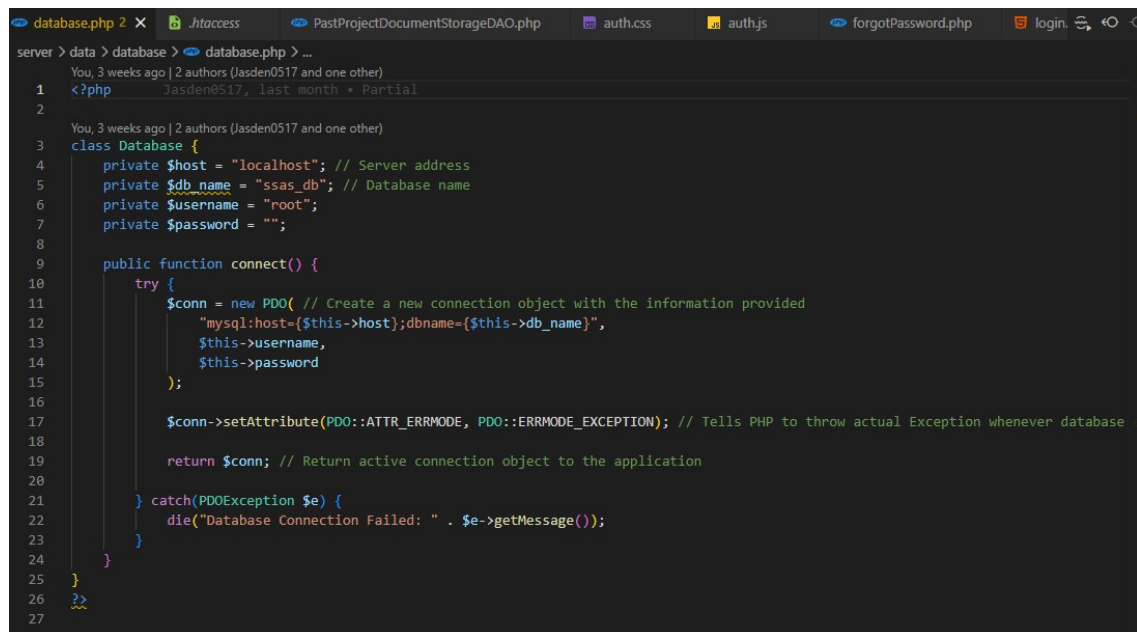
Figure 5.1: Apache .htaccess file

Database Connection and Error Handling

Data persistence is handled by a MySQL Relational Database Management System (RDBMS) running on default port 3306. The system connects to the `ssas_db` schema via PHP

Data Objects (PDO), entirely bypassing legacy mysqli drivers to ensure maximum security against SQL injection.

Within the server/data/database/database.php script, the connection is instantiated per Data Access Object (DAO) to prevent monolithic bottleneck. Crucially, the PDO connection is explicitly configured to enforce strict error handling by setting PDO::ATTR_ERRMODE to PDO::ERRMODE_EXCEPTION. This guarantees that any failed database transaction (such as a concurrent double-booking attempt) throws an immediate, catchable exception, satisfying Reliability NFR1.



```

1  <?php
2
3  class Database {
4      private $host = "localhost"; // Server address
5      private $db_name = "ssas_db"; // Database name
6      private $username = "root";
7      private $password = "";
8
9      public function connect() {
10         try {
11             $conn = new PDO( // Create a new connection object with the information provided
12                 "mysql:host={$this->host};dbname={$this->db_name}",
13                 $this->username,
14                 $this->password
15             );
16
17             $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION); // Tells PHP to throw actual Exception whenever database
18
19             return $conn; // Return active connection object to the application
20
21         } catch(PDOException $e) {
22             die("Database Connection Failed: " . $e->getMessage());
23         }
24     }
25 }
26
27

```

Figure 5.2: database.php

Application-Level Security and Temporal Configurations

Rather than relying on global php.ini files, the SSAS enforces its security and temporal constraints directly within the application's business logic, ensuring the code remains highly portable:

- Session Timeout Enforcement:** To satisfy Security NFR 4 (the 15-minute auto-logout constraint), the SessionManager.php script utilizes a sliding/rolling expiration window. A constant SESSION_TIMEOUT = 900 (representing 900 seconds) is defined. On every request, the system compares the current time() against the \$_SESSION["lastActivity"] variable. If the threshold is exceeded, the session is programmatically destroyed.

```
class SessionManager {  
  
    /*  
    |-----  
    | Session Timeout Configuration  
    |-----  
    */  
  
    private const SESSION_TIMEOUT =  
        900; // 15 minutes
```

Figure 5.3 SessionManager.php timeout configuration

```
private static function validateSessionTimeout()  
  
    /*  
    |-----  
    | Skip if User Not Logged In  
    |-----  
    */  
  
    if (  
        !isset($_SESSION["userID"])  
    ) {  
        return;  
    }  
  
    /*  
    |-----  
    | Timeout Validation  
    |-----  
    */  
  
    if (  
        isset($_SESSION["lastActivity"])  
    ) {  
        $inactiveDuration =  
            time() - $_SESSION["lastActivity"];  
        if ($inactiveDuration > self::SESSION_TIMEOUT ) {  
            self::destroySession();  
            self::redirectToLogin(  
                "error",  
                "Session expired"  
            );  
            exit();  
        }  
    }  
}
```

Figure 5.4 SessionManager.php Compare Current Time with Last Activity

- **Immutable Timezone Syncing:** To prevent desynchronisation during Just-In-Time (JIT) evaluations, specific temporal services (such as the TemporalPhaseEngine.php) independently declare a `SYSTEM_TIMEZONE = "Asia/Kuala_Lumpur"` constant. This ensures backend timeline locks are evaluated using the exact same Malaysia standard time, regardless of global server configurations.

```
private const SYSTEM_TIMEZONE =  
    "Asia/Kuala_Lumpur";
```

Figure 5.5: Defining System Timezone

- **Payload Capacity Limits:** While the .htaccess file allows up to 50MB, the system explicitly enforces the Capacity NFRs via the ProposalStorageDAO.php. A mathematical constant, `MAX_PROPOSAL_SIZE = 5242880` (exactly 5.0 MB in bytes), is defined within the class. Any uploaded FYP proposal exceeding this exact byte count is programmatically intercepted before database commitment.

```
private const MAX_PROPOSAL_SIZE = 5242880; // 5MB in bytes
```

Figure 5.6: Defining proposal size explicitly

5.1.2 Implementation of Core Design Patterns

This subsection explicitly demonstrates how the theoretical software architectures designed in Chapter 4 were practically engineered within the PHP backend. By utilising advanced Object-Oriented Design (OOD) patterns, the system avoids procedural “spaghetti code” and ensures modular maintainability

1. Observer Pattern (Supervisor Availability & Status Module) .

To resolve the “Static Data Trap” identified in legacy workflow, the system requires immediate UI synchronisation whenever a supervisor’s allocation capacity changes. This requirement was fulfilled by engineering a strict Observer Design Pattern within the server/business/services/SupervisorAvailabilityService.php file.

The architecture physically decouples the backend state management (The Subject) from the frontend display logic (The observer), establishing a “Single Source of Truth” for all status badges.

Implementation of the Subject (Publisher)

The AllocationRegistry class acts as the Subject. It is responsible for tracking supervisor capacity and maintaining a dynamic array of subscribed observers. When the system processes a capacity change, it triggers notifyObservers(), broadcasting the new payload to all attached UI components simultaneously.

```
class AllocationRegistry { // Calculate and broadcast supervisor's real time availability

    private const ONLINE_THRESHOLD_SECONDS = 900;

    private $observers = [];

    public function attach(AvailabilityObserver $observer) {
        $this->observers[] =
            $observer;
    }

    public function detach(AvailabilityObserver $observer) {
        $this->observers =
            array_filter($this->observers, function ($registeredObserver) use ($observer) {
                return $registeredObserver !== $observer;
            });
    }

    public function saveAllocationStatus($supervisor) {
        $payload = $this->buildAvailabilityPayload($supervisor);
        $this->notifyObservers($payload);
        return $payload;
    }

    private function notifyObservers($payload) {
        foreach ($this->observers as $observer) {
            $observer->update($payload);
        }
    }
}
```

Figure 5.7 Implementation for AllocationRegistry class

Implementation of the Concrete Observer (Subscriber)

To standardise the communication protocol, the system utilises an AvailabilityObserver interface. The SupervisorAvailabilityUI class implements this interface, acting as the Concrete Observer responsible for translating the raw backend payload into the actionable UI states required by the frontend.

```
interface AvailabilityObserver {
    public function update($availabilityPayload);
}

You, 10 minutes ago | 2 authors (Jasden0517 and one other)
class SupervisorAvailabilityUI implements AvailabilityObserver {

    private $payload = [];

    public function update($availabilityPayload) {
        $isOnline = ($availabilityPayload["availabilityStatus"] ?? "Offline") === "Online";
        $isQuotaOpen = ($availabilityPayload["quotaStatus"] ?? "Full") === "Available";
        $this->payload = array_merge($availabilityPayload, [
            "availabilityClass" => $isOnline ? "online" : "offline",
            "quotaClass" => $isQuotaOpen ? "available" : "full",
            "canApply" => $this->toggleApplyButton($isQuotaOpen),
            "buttonLabel" => $this->buttonLabel($isQuotaOpen),
            "message" => $this->statusMessage($isQuotaOpen)
        ]);
    }

    public function calculateQuota() {
        return
            ($this->payload["activeStudents"] ?? 0) .
            "/" .
            ($this->payload["maxSlots"] ?? 0) .
            " slots taken";
    }

    public function toggleApplyButton($isQuotaOpen) {
        return (bool) $isQuotaOpen;
    }
}
```

Figure 5.7 Implementation for AvailabilityObserver Interface and SupervisorAvailabilityUI

Fulfillment of Functional Requirements and Constraints

This pattern directly satisfies the module's core Functional Requirements.

- 1. Dynamic Status Badges (FR 2.3 & 2.4):** The `update()` method dynamically assigns the CSS `quotaClass` (e.g., `available` or `full`). The PHP view engine reads this class to render the corresponding Green or Red visual badges on the Discovery list.
- 2. Submission Blocking (FR 3.2):** The `toggleApplyButton()` helper method returns a strict boolean state. If a supervisor reaches their predefined administrative limit, `$isQuotaOpen` evaluates to false. The frontend utilizes the resulting `canApply` boolean to physically disable and grey out the "Apply" button, guaranteeing that over-booking is mathematically impossible.
- 3. Data Integrity:** Rather than relying on static database flags, the raw quota figures injected into the `AllocationRegistry` are derived from live `COUNT(DISTINCT AR.allocationID)` and `COALESCE()` SQL queries, cross-referencing the `ALLOCATION_RECORD` and `QUOTA_CONFIGURATION` tables. This ensures the observers are always reacting to the most accurate, real-time database state.

2. Template Method Pattern (Discovery & Search Module) .

To resolve the “Discovery Difficulty” identified during requirement analysis, the system implements a dynamic filtering and recommendation engine. Because both manual searches and automated recommendations share a similar sequence of database operations, the Template Method Design Pattern was engineered to standardise the workflow.

This pattern defines the absolute “skeleton” of the search algorithm in a base class, while allowing subclasses to safely override specific mathematical matching and sorting behaviours without altering the overarching architecture.

The Abstract Template (The Algorithm Skeleton)

The DiscoveryEngine abstract class serves as the foundational template. It defines the executeSearch() method, which orchestrates a strict sequence of operations: fetching records, attaching tags, filtering availability, applying the specific matching logic, ranking, and rendering. By enforcing this strict sequence, the system mathematically guarantees that all search types perform the mandatory capacity checks before returning results to the student.

```
abstract class DiscoveryEngine { // Abstract class that defines structure for discovery method

    protected $supervisorDAO;
    protected $tagDAO;
    protected $availabilityService;

    public function __construct() {
        $this->supervisorDAO = new SupervisorDAO();
        $this->tagDAO = new TagDAO();
        $this->availabilityService = new SupervisorAvailabilityService();
    }

    // Calls multiple methods to transform raw data into polished result
    public function executeSearch($context) {
        $supervisors = $this->fetchActiveSupervisors(); // Get a raw list of supervisors and decorates them with their availability s
        $supervisors = $this->attachTagPayload($supervisors); // Add supervisor's research interest tag to their profile
        $supervisors = $this->filterSlotAvailability($supervisors, $context); // Filter student's availability status choice
        $supervisors = $this->applyMatchingLogic($supervisors, $context); // An abstract method
        $supervisors = $this->rankResults($supervisors, $context); // sorts the filtered list
        return $this->renderResults($supervisors, $context); // Returns the final results
    }

    abstract protected function applyMatchingLogic($supervisors, $context); // Used for different search types, including manual and
```

Figure 5.8 Implementation for DiscoveryEngine

The Concrete Subclass (Recommendation Math and Hook Overrides)

To satisfy the requirement for an intelligent matchmaking feature (FR 2.2), the RecommendationProcessor class extends the DiscoveryEngine. It provides a concrete implementation of the applyMatchingLogic() method, utilising PHP's array_intersect to mathematically calculate the exact overlap between the student's saved research tags and the supervisor's expertise tags

Furthermore, this subclass overrides the parent class's rankResults() and renderResults() hooks. It injects a custom usort function to order the results by highest matchScore, and utilises array_slice to strictly limit the final payload to three records, fulfilling the "Top 3" UI constraint (FR 2.5).

```
class RecommendationProcessor extends DiscoveryEngine {

    protected function applyMatchingLogic($supervisors, $context) {
        // Check if student has any tags
        $studentTagIDs = $this->tagDAO->getStudentTagIDs($context["studentID"] ?? "");

        if (empty($studentTagIDs)) { return []; }

        $matched = [];

        // Calculate supervisor limit
        foreach ($supervisors as $supervisor) {
            if (($supervisor["quotaStatus"] ?? "Full") != "Available") { // Skip if supervisor is not available
                continue;
            }

            // Find overlapping tags between student and supervisor
            $matchScore = $this->intersects($studentTagIDs, $supervisor["tagIDs"]);

            if ($matchScore <= 0) {
                continue;
            }

            $supervisor["matchScore"] = $matchScore;
            $matched[] = $supervisor;
        }

        return $matched;
    }

    // Sorts overlapping count DESCENDING, limit output to maximum of 3 elements
    protected function rankResults($supervisors, $context) {
        usort($supervisors, function ($first, $second) {
            if ($first["matchScore"] != $second["matchScore"]) {
                return $second["matchScore"] <=> $first["matchScore"]; // Sort in descending order
            }

            return strcmp($first["fullName"], $second["fullName"]);
        });
        return $supervisors;
    }

    protected function renderResults($supervisors, $context) {
        return array_slice(array_values($supervisors), 0, 3); // Ensure only top 3 are returned to the UI
    }
}
```

Figure 5.9 Implementation for RecommendationProcessor

Architectural Orchestration

To ensure the presentation layer remains completely decoupled from this complex mathematics, the SupervisorDiscoveryService acts as a facade. When the frontend requests a search or recommendation, this service dynamically instantiates either the ManualSearchProcessor or the RecommendationProcessor and calls the unified executeSearch() template method, adhering strictly to the Open-Closed Principle for future algorithm expansion.

3. State Pattern (Request & Proposal Module).

The request & Proposal module manages a highly complex, non-linear lifecycle. A single submitted project proposal can transition into multiple states based on supervisor actions (Accept/Reject) or automated system triggers (72-hour Timeouts). To avoid “Conditional Logic Bloat” within the main controller, the system implements the State Design Pattern.

This architecture delegates dynamic behaviour to discrete state objects, ensuring that transitions such as the automated withdrawal of pending requests when a student is accepted are handled polymorphically without requiring massive, unmaintainable if/else control structures.

The State Interface and Terminal Abstraction

The ApplicationState interface defines the strict operational contract for all possible proposal statuses. To adhere to the DRY (Don’t Repeat Yourself) principle, the system introduces a TerminalApplicationState abstract class. Because statuses including “Accepted”, “Rejected”, “Timeout”, “withdrawn” and “auto-allocated” are absorbing states (meaning the decision is final and no further transitions are allowed), they all inherit from this abstract parent. The parent intercepts any subsequent method calls and returns a unified lockedResponse(), physically preventing out-of-sequence system actions.

```
interface ApplicationState {  
    public function accept(FYPApplication $application, $comment);  
    public function reject(FYPApplication $application, $comment);  
    public function timeout(FYPApplication $application, $comment);  
    public function withdraw(FYPApplication $application, $comment);  
    public function autoAllocate(FYPApplication $application, $comment);  
}
```

Figure 5.10 Implementation of ApplicationState Interface

```

abstract class TerminalApplicationState implements ApplicationState {
    public function accept(FYPApplication $application, $comment) {
        return $this->lockedResponse();
    }

    public function reject(FYPApplication $application, $comment) {
        return $this->lockedResponse();
    }

    public function timeout(FYPApplication $application, $comment) {
        return $this->lockedResponse();
    }

    public function withdraw(FYPApplication $application, $comment) {
        return $this->lockedResponse();
    }

    public function autoAllocate(FYPApplication $application, $comment) {
        return $this->lockedResponse();
    }

    protected function lockedResponse() {
        return [
            "success" => false,
            "message" => "Decision already processed for this proposal. A new decision is only available after the student submits a new proposal."
        ];
    }
}

```

Figure 5.11 Implementation of TerminalApplicationState abstract class

```

// Concrete terminal states are clean, empty subclasses inheriting the locks
class AcceptedState extends TerminalApplicationState {}

```

Figure 5.12 Absorbing states inheriting TerminalApplicationState

The Active State and Context Delegation

Unlike the terminal states, the PendingState class implements the dynamic logic required to transition the application. When a supervisor clicks “Accept”, the PendingState mutates the context entity (FYPApplication) by executing the changestate() command.

The FYPApplication itself acts as the Context Entity. It holds the current state as an internal attribute and strictly delegates all operational commands (like accept()) to that state object, remaining entirely decoupled from the transition logic.

```
class PendingState implements ApplicationState {  
  
    public function accept(FYPApplication $application, $comment) {  
  
        $application->setComment($comment);  
        $application->changeState(new AcceptedState());  
  
        return [  
            "success" => true,  
            "decisionStatus" => "Accepted",  
            "comment" => $application->getComment()  
        ];  
    }  
  
    Jasden0517, last month + Partial  
    public function reject(FYPApplication $application, $comment) {  
  
        $application->setComment($comment);  
        $application->changeState(new RejectedState());  
  
        return [  
            "success" => true,  
            "decisionStatus" => "Rejected",  
            "comment" => $application->getComment()  
        ];  
    }  
  
    public function timeout(FYPApplication $application, $comment) {  
  
        $application->setComment($comment);  
        $application->changeState(new TimeoutState());  
  
        return [  
            "success" => true,  
            "decisionStatus" => "Rejected-Timeout",  
            "comment" => $application->getComment()  
        ];  
    }  
  
    public function withdraw(FYPApplication $application, $comment) {  
  
        $application->setComment($comment);  
        $application->changeState(new WithdrawnState());  
  
        return [  
            "success" => true,  
            "decisionStatus" => "Withdrawn",  
            "comment" => $application->getComment()  
        ];  
    }  
  
    public function autoAllocate(FYPApplication $application, $comment)  
  
        $application->setComment($comment);  
        $application->changeState(new AutoAllocatedState());  
  
        return [  
            "success" => true,  
            "decisionStatus" => "Auto-Allocated",  
            "comment" => $application->getComment()  
        ];  
    }  
}
```

Figure 5.13: Implementation of PendingState class


```
public function changeState(ApplicationState $state) {  
    $this->state = $state;  
}  
  
public function accept($comment) {  
    return $this->state->accept($this, $comment);  
}
```

Figure 5.14: Implementation of FYPAApplication class

Architectural Orchestration and Rehydration

Since PHP operates within a stateless request-response lifecycle, the transient FYPAApplication object is not durably persisted in active memory. To bridge the Object-Oriented design with the relational database, the RequestDecisionService functions as the orchestrator.

Before processing a decision, the service retrieves the plaintext status string (e.g., “Pending”) from the database and utilises a stateFromStatus() factory method to rehydrate the exact GoF State object. The FYPAApplication is then constructed with its initial state, allowing the polymorphic execution of the accept() or reject() methods before committing the final mutated status back to the SQL data layer.

4. Facade Pattern (Student Profile Module).

The Student Profile module prioritises strict data integrity, memory optimisation, and rigorous input validation over complex behavioural state transitions. To satisfy the requirements outlined in UC400 without cluttering the presentation layer or bloating the core entity objects, the system implements the Facade Design Pattern

This architecture delegates the complex orchestration of multiple database operations and file-handling logic to a centralised service layer, shielding the boundary layer from the intricate data formatting rules.

The Logic Controller (Facade Composition)

The StudentProfileFacade class serves as the unified entry point for profile updates. Rather than forcing the frontend controller to interact with multiple distinct data services, the Facade's constructor instantiates and composes three separate collaborators: StudentProfileDAO, TagDAO, and ImageStorageDAO. It then exposes a single, simplified updateProfile() method that coordinates existence checks, data validation, file storage, and database rollbacks within a single transaction sequence.

```
// Subsystem collaborators
private $studentProfileDAO;
private $tagDAO;
private $imageStorageDAO;

public function __construct() {
    // Composition of underlying data access objects
    $this->studentProfileDAO = new StudentProfileDAO();
    $this->tagDAO = new TagDAO();
    $this->imageStorageDAO = new ImageStorageDAO();
}
```

Figure 5.15: StudentProfileFacade subsystem collaborators

```

// Unified Facade method: Orchestrates the entire update lifecycle
public function updateProfile($studentID, $input, $avatarFile) {}
$studentID = trim((string) $studentID);
$profile = $this->studentProfileDAO->getStudentProfile($studentID);

if (!$profile) {
    return $this->failure("Student profile was not found.");
}

$contactNumber = trim((string) ($input["contactNumber"] ?? ""));
$personalBio = trim((string) ($input["personalBio"] ?? ""));
$linkedinURL = trim((string) ($input["linkedinURL"] ?? ""));
$githubURL = trim((string) ($input["githubURL"] ?? ""));
$portfolioURL = trim((string) ($input["portfolioURL"] ?? ""));
$tagIDs = $this->normaliseTagIDs($input["interestTags"] ?? []);

if (strlen($personalBio) > self::MAX_BIO_LENGTH) {
    return $this->failure("Validation Error - Personal Bio cannot exceed 500 characters.");
}

if (count($tagIDs) > self::MAX_TAGS) {
    return $this->failure("You can select a minimum of 1 and a maximum of 5 research interests . Please remove one to add another.");
}

if (empty($tagIDs)) {
    return $this->failure("Please select at least one research interest.");
}

if (!$this->tagDAO->tagIDsExist($tagIDs)) {
    return $this->failure("Save failed. Please ensure all selected research interests are valid.");
}

if (!$this->isValidOptionalURL($linkedinURL) ||
    !$this->isValidOptionalURL($githubURL) ||
    !$this->isValidOptionalURL($portfolioURL)) {
    return $this->failure("Save failed. Please ensure all links begin with a valid URL.");
}

$avatarPath = null;
$oldAvatarPath = $profile["profilePhotoPath"] ?? "";

if ($this->hasUploadedAvatar($avatarFile)) {
    if ((int) $avatarFile["size"] > self::MAX_AVATAR_SIZE) {
        return $this->failure("Upload failed. Please ensure your profile picture is in JPG or PNG format and does not exceed 2.0MB.");
    }

    $avatarResult = $this->imageStorageDAO->storeProfilePhoto($avatarFile, $studentID);

    if (!$avatarResult["success"] || $avatarResult["path"] === null) {
        return $this->failure("Upload failed. Please ensure your profile picture is in JPG or PNG format and does not exceed 2.0MB.");
    }

    $avatarPath = $avatarResult["path"];
}

$update = $this->studentProfileDAO->updateStudentProfile(
    $studentID,
    [
        "contactNumber" => $contactNumber,
        "personalBio" => $personalBio,
        "linkedinURL" => $linkedinURL,
        "githubURL" => $githubURL,
        "portfolioURL" => $portfolioURL
    ],
    $tagIDs,
    $avatarPath
);

if (!$update) {
    if ($avatarPath !== null) {
        $this->imageStorageDAO->deleteStoredImage($avatarPath);
    }
    return $this->failure("Save failed. Please ensure all links begin with a valid URL.");
}

if ($avatarPath !== null) {
    $this->imageStorageDAO->deleteStoredImage($oldAvatarPath);
}

return $this->success("Your profile has been successfully updated.");
}

```

Figure 5.16: StudentProfileFacade updateProfile() method

Strict Validation and Defense in Depth

Before any database mutation occurs, the Facade acts as a strict validation gatekeeper, programmatically enforcing the Functional Constraints designed in Chapter 4. It explicitly defines constants for maximum lengths and utilises Regular Expressions (Regex) to ensure data sanitisation.

Crucially, the architecture employs a “Defense in Depth” strategy for file uploads. The Facade executes a lightweight, “fail-fast” byte-size evaluation (MAX_AVATAR_SIZE) against the \$avatarFile. If the file passes, it is then delegated to the underlying ImageStorageDAO which independently re-verifies the size and executes deep MIME-type and extension validation. This dual-layered security prevents malicious large payloads from consuming unnecessary server processing power.

```
// Physical enforcement of Functional Constraint
private const MAX_BIO_LENGTH = 500;
private const MAX_TAGS = 5;
private const MAX_AVATAR_SIZE = 2097152; // 2.0 MB payload limit
private const VALID_URL_PATTERN = "/^https?:\\/\\/([\\s\\/$.\\?#].[^\\s]*)$/i";
```

Figure 5.17: First Layer Defense

```
// Constraint C1: Personal Bio limitation
if (strlen($personalBio) > self::MAX_BIO_LENGTH) {
    return $this->failure("Validation Error - Personal Bio cannot exceed 500 characters.");
}

// FR 2.2: Tag Array limitation
if (count($tagIDs) > self::MAX_TAGS) {
    return $this->failure("You can select a minimum of 1 and a maximum of 5 research interests . Please remove one to add another.");
}

// Defense in Depth: Fail-fast file size check before DAO delegation
if ($this->hasUploadedAvatar($avatarFile)) {
    if ((int) $avatarFile["size"] > self::MAX_AVATAR_SIZE) {
        return $this->failure("Upload failed. Please ensure your profile picture is in JPG or PNG format and does not exceed 2.0MB.");
    }

    $avatarResult = $this->imageStorageDAO->storeProfilePhoto($avatarFile,$studentID);

    if (!$avatarResult["success"] || $avatarResult["path"] === null) {
        return $this->failure("Upload failed. Please ensure your profile picture is in JPG or PNG format and does not exceed 2.0MB.");
    }

    $avatarPath = $avatarResult["path"];
    // Jasden0517, last month * Partial
}
```

```
// Constraint C2: Regex evaluation for external links
private function isValidOptionalURL($url) {
    return $url === "" ||
        preg_match(self::VALID_URL_PATTERN, $url) === 1;
}
```

Figure 5.18: Second Layer Defense

Subsystem Entities and Data Segregation

To satisfy the sub-millisecond execution requirements of the Supervisor Recommendation algorithm, the Facade fundamentally segregates data handling. The core Student class remains a lightweight object, containing only the attributes critical for the mathematical matching engine. Conversely, heavier, structurally expensive data, such as the personal bio string and formatted URLs, does not exist as a stateful object in server memory. Instead, the StudentProfileFacade retrieves and passes this extended profile data as raw associative arrays directly from the data layer to the presentation layer. This stateless approach prevents memory bloat during background operations.

5. Proxy Pattern (Student Review Module).

The Student Review module requires a strict balance between administrative traceability and frontend privacy. The system must permanently bind the author's identity to the review to prevent spam and allow for administrative audits, while conditionally masking it from public view based on a user-defined boolean. To execute this Data Masking constraint (Constraint C2 of UC500), the system implements a Protection Proxy.

By intercepting read requests, the Proxy ensures that the public frontend never directly accesses the sensitive student identifier, preventing identity leaks while maintaining full database integrity.

The Logic Controller (Validation Gatekeeper)

Before a review can be masked and saved, the StudentReviewService acts as the primary validation gatekeeper, enforcing the module's Functional Constraints. The controller intercepts the submission payload and executes a dependency check against the allocationID to physically prevent duplicate review entries. Furthermore, it enforces the 1000-character payload limit on the text feedback and verifies the mandatory 1 to 5 integer star rating.

```
// Dependency check: prevents duplicate reviews for the same allocation
if ($this->reviewDAO->reviewExistsForAllocation($allocationID)) {
    return $this->failure("You have already submitted an evaluation for your supervisor this semester.");
}

// Constraint: mandatory 1 to 5 rating scale
if ($starRating < 1 || $starRating > 5) {
    return $this->failure("Submission failed. You must select a star rating (1-5) to submit a review. Text feedback is optional.");
}

// Constraint C1: Feedback payload limit
if (strlen($textFeedback) > self::MAX_FEEDBACK_LENGTH) {
    return $this->failure("Submission failed. Feedback cannot exceed 1000 characters.");
}
```

Figure 5.19: Functional Constraint in StudentReviewService

The Interface and Real Subject

To handle data sanitisation without relying on procedural if/else statements in the frontend view, the system utilises a strict Gang-of-Four Proxy architecture. The ReviewRecordView interface defines the display() contract. The raw database payload, which contains the highly sensitive trueStudentName and trueStudentID, is wrapped in the SupervisorReviewRecord class, acting as the Real Subject.

```
interface ReviewRecordView {  
    public function display();  
}
```

Figure 5.20: Implementation of ReviewRecordView Interface

```
class SupervisorReviewRecord implements ReviewRecordView {  
    private $review;  
  
    public function __construct($review) {  
        $this->review = $review;  
    }  
  
    public function display() {  
        return [  
            "reviewID" => $this->review["reviewID"],  
            "authorName" => $this->review["trueStudentName"] ?? "Student",  
            "authorID" => $this->review["trueStudentID"] ?? "",  
            "starRating" => (int) $this->review["starRating"],  
            "textFeedback" => $this->review["textFeedback"] ?? "",  
            "isAnonymous" => (bool) ($this->review["isAnonymous"] ?? false)  
        ];  
    }  
}
```

Figure 5.21: Implementation of SupervisorReviewRecord class

The Proxy Implementation (Data Masking)

To mask the data securely, the `AnonymousReviewProxy` class also implements the `ReviewRecordView` interface. It composes the Real Subject in its constructor. When the `display()` method is called by the frontend, the Proxy first retrieves the authentic payload from the Real Subject. It then evaluates the `isAnonymous` flag. If evaluated as true, the Proxy actively strips the sensitive identifiers and overrides the output with the string “Anonymous Student”

```
class AnonymousReviewProxy implements ReviewRecordView {  
  
    private $reviewRecord;  
  
    public function __construct(ReviewRecordView $reviewRecord) {  
        $this->reviewRecord = $reviewRecord;  
    }  
  
    public function display() {  
        $payload = $this->reviewRecord->display();  
        if ($payload["isAnonymous"]) {  
            $payload["authorName"] = "Anonymous Student";  
            $payload["authorID"] = "";  
        }  
  
        return $payload;  
    }  
}
```

Figure 5.22: Implementation of `AnonymousReviewProxy` class

Proxy Instantiation and Orchestration

The instantiation of these wrappers occurs dynamically when the presentation layer requests the data. Within the `StudentReviewService`, the `getSanitizedReviewForSupervisor()` method retrieves the raw SQL rows. It utilises `array_map()` to wrap each row first in the `SupervisorReviewRecord`, and immediately wraps that object inside the `AnonymousReviewProxy`. Only the completely sanitised output of the Proxy’s `display()` method is finally returned to the client, guaranteeing that unauthorised data exposure is structurally impossible.


```
public function getSanitizedReviewsForSupervisor($supervisorID) {  
    $reviews = $this->reviewDAO->getSanitizedReviewsForSupervisor($supervisorID);  
  
    return array_map(  
        function ($review) {  
            return (new AnonymousReviewProxy( // Decides which part of raw data the user is allowed to see  
                new SupervisorReviewRecord($review) // Holds all of the raw data  
            ))->display();  
        },  
        $reviews  
    );  
}
```

Figure 5.23: getSanitizedReviewSupervisor Function Implementation

6. Singleton Pattern (Timeline & Milestone Module).

The Timeline & Milestone module serves as the primary temporal security mechanism for the entire application. To prevent malicious client-side manipulation of academic deadlines, the system must utilise a centralised, authoritative server-side clock. The Singleton Design Pattern is implemented to mathematically guarantee that only one instance of the time-evaluation engine exists within the system memory. This provides a single, immutable source of truth for the server's timestamp, preventing cross-module concurrency conflicts when external services query the global phase status.

The Singleton Structure

The TemporalPhaseEngine encapsulates the logic for the temporal phase evaluation. It adheres strictly to the Gang of Four (GoF) Singleton structure by declaring a private static \$instance variable, enforcing a private __construct() to prevent external instantiation, and exposing a lazy-initiated public static function getInstance() method.

```
private const SYSTEM_TIMEZONE = "Asia/Kuala_Lumpur";
private static $instance = null;
private $conn;
private $requestDAO;

private function __construct() {
    date_default_timezone_set(self::SYSTEM_TIMEZONE);
    $database = new Database();
    $this->conn = $database->connect();
    $this->requestDAO = new RequestDAO();
    $this->ensureSystemPhaseTable();
}

public static function getInstance() {
    if (self::$instance === null) {
        self::$instance = new TemporalPhaseEngine();
    }

    return self::$instance;
}
```

Figure 5.24: Implementation of Singleton TemporalPhaseEngine

Just-In-Time (JIT) Evaluation and Formatting

Rather than relying on resource-intensive batch-processing Cron jobs, the engine executes a Just-In-Time (JIT) evaluation. It computes the active operational phase dynamically by extracting the server's UTC timestamp. The remaining phase allowance is calculated using strict epoch mathematical subtraction and chunked via `intdiv()` into a uniform "Days:Hours:Minutes" string payload.

```
// Epoch subtraction using the server's immutable timestamp
$remainingSeconds =
    max(0, $this->toDateTime($activePhase["endTimeStamp"])->getTimestamp() - $serverNow->getTimestamp());
```

Figure 5.25: Epoch Subtraction

```
// Formats output into strictly chunked strings
private function formatRemaining($seconds) {
    $seconds = max(0, (int) $seconds);
    $days = intdiv($seconds, 86400);
    $hours = intdiv($seconds % 86400, 3600);
    $minutes = intdiv($seconds % 3600, 60);

    return
        str_pad((string) $days, 2, "0", STR_PAD_LEFT) . "d "
        . str_pad((string) $hours, 2, "0", STR_PAD_LEFT) . "h "
        . str_pad((string) $minutes, 2, "0", STR_PAD_LEFT) . "m";
}
```

Figure 5.26: Output Formatting

Cross-Module Dependency (Security Lock)

External controllers maintain a critical dependency on this Singleton to enforce business rules. For example, the `RequestWorkflowManager` does not calculate time itself. Before allowing a student to submit a proposal, it queries the Singleton's `isSubmissionPhaseActive()` boolean. This architecture ensures that POST privileges are instantly revoked the exact millisecond the Submission Phase expires, making late submissions structurally impossible.

```

public function __construct() {
    $this->requestDAO = new RequestDAO();
    $this->supervisorProfileService = new SupervisorProfileService();
    $this->proposalStorageDAO = new ProposalStorageDAO();
    $this->allocationWindowService = new AllocationWindowService();
    $this->availabilityService = new SupervisorAvailabilityService();

    // Accesses the global temporal engine Singleton
    $this->temporalPhaseEngine = TemporalPhaseEngine::getInstance();
}

```

Figure 5.27: Accessing Global TemporalPhaseEngine

```

/*
|-----|
| Server-Side Phase Validation |
|-----|
| Uses the singleton temporal engine so proposal POST access follows the |
| central server timeline, not a browser-controlled clock. |
*/
if ($requestedProposal === null && !$this->temporalPhaseEngine->isSubmissionPhaseActive()) {
    return $this->failure("Submission phase has expired. Proposal submission is currently locked.");
}

```

Figure 5.28: Active Security Lock that rejects POST requests if phase expired

The Boundary: Client-Side Polling

Since PHP is a synchronous backend language, it cannot execute a continuous UI countdown natively. The architectural boundary shifts to client-side Javascript (student.js). A local JavaScript interval (renderCountdown) updates the visual timer every 1,000ms. However, to prevent browser desynchronisation (such as a browser tab going to sleep), a secondary polling loop queries the Singleton endpoint every 60,000ms (refreshTimeline). This ensures the client UI is continuously re-synchronised with the authoritative server clock.

```

// Re-syncs the UI with the authoritative server clock
async function refreshTimeline() {
    try {
        const response = await fetch("../server/application/student/getTimelineStatus.php", { credentials: "same-origin" });
        if (response.ok) renderPayload(await response.json());
    } catch (error) { /* Silent fail */ }
}

// Local 1000ms visual tick
renderCountdown();
setInterval(renderCountdown, 1000);
setInterval(refreshTimeline, 60000); // Authoritative server poll to prevent browser desynchronization

```

Figure 5.29: refreshTimeline() Function Implementation

5.1.3 Implementation of Business Logic and Constraint

This subsection demonstrates how the system's most critical Functional Requirements (FRs) and business constraints were physically implemented using PHP and SQL. Rather than relying entirely on frontend validation, the architecture enforces these rules strictly at the backend data layer to prevent exploitation.

1. Concurrency Limits (Anti-Spam Mechanism)

To prevent system overload and stop students from spamming multiple faculty members simultaneously, the Request & Proposal module enforces a strict concurrency threshold (FR 2.2). A student is mathematically restricted to a maximum of three active pending requests.

This constraint is enforced centrally within the RequestWorkflowManager. This class defines a static mathematical constant `MAX_PENDING_REQUESTS = 3`. During the `submitProposal()` execution, it delegates a counting query to the Data Access Object (DAO) before processing the payload. If the integer returned meets or exceeds the threshold, the POST request is physically rejected.

```
// Defining strict concurrency threshold  
private const MAX_PENDING_REQUESTS = 3;
```

Figure 5.30: MAX_PENDING_REQUESTS constant in RequestWorkflowManager class

```
/*  
|-----  
| Concurrency Limit Validation  
|-----  
| Enforces the maximum of three concurrent pending proposal requests.  
*/  
// Validates active requests against threshold  
if (  
    $requestedProposal === null &&  
    $this->requestDAO->countPendingRequestsByStudent($studentID)  
    >= self::MAX_PENDING_REQUESTS  
) {  
    return $this->failure(  
        "You may only have up to 3 active pending proposals."  
    );  
}
```

Figure 5.31: Pending Request Limit Validation in submitProposal() within RequestWorkflowmanager

To ensure accuracy, the underlying RequestDAO executes an explicit COUNT(*) SQL query. It intentionally evaluates both 'Pending' and 'Proposal Requested' states to capture the true volume of unresolved interactions.

```
public function countPendingRequestsByStudent($studentID) {  
    $query = "  
        SELECT COUNT(*)  
        FROM APPLICATION_REQUEST  
        WHERE studentID = :studentID  
        AND decisionStatus IN ('Pending', 'Proposal Requested')  
    ";  
  
    $statement = $this->conn->prepare($query);  
  
    $statement->bindParam(":studentID", $studentID);  
  
    $statement->execute();  
  
    return (int) $statement->fetchColumn();  
}
```

Figure 5.32: Count Student Pending Request Function in RequestDAO

2. Automated Conflict Resolution (Auto-Withdrawal)

The system must guarantee a strict 1-to-1 final allocation. When a supervisor clicks “Accept” on a proposal, the system must autonomously locate and withdraw any other pending requests the student currently holds to free up those slots for other students (FR 2.4).

Rather than pulling all requests into PHP memory and looping through them individually, the RequestDAO executes a highly optimised bulk SQL UPDATE statement. Crucially, this operation is bound within the same database transaction as the acceptance itself. This guarantees ACID compliance, if the auto-withdrawal fails, the acceptance is rolled back, preventing orphaned data.

```

class RequestDAO {
    public function processSupervisorDecision(

        if ($decisionStatus === "Accepted") {

            if (!$hasExistingSupervisorAllocation) {

                $insertQuery = "
                    INSERT INTO ALLOCATION_RECORD
                    (
                        studentID,
                        supervisorID,
                        allocationDate,
                        allocationMethod
                    )
                    VALUES
                    (
                        :studentID,
                        :supervisorID,
                        NOW(),
                        'Approved Request'
                    )
                ";

                $insertStatement = $this->conn->prepare($insertQuery);
                $insertStatement->bindParam(":studentID", $request["studentID"]);
                $insertStatement->bindParam(":supervisorID", $supervisorID);
                $insertStatement->execute();
            }

            // Bulk UPDATE query targets all OTHER pending requests for this student
            $withdrawQuery = "
                UPDATE APPLICATION_REQUEST
                SET decisionStatus = 'Withdrawn',
                    supervisorComment = 'Automatically withdrawn because another proposal was accepted.'
                WHERE studentID = :studentID
                AND requestID <> :requestID
                AND decisionStatus = 'Pending'
            ";

            $withdrawStatement = $this->conn->prepare($withdrawQuery);
            $withdrawStatement->bindParam(":studentID", $request["studentID"]);
            $withdrawStatement->bindParam(":requestID", $requestID, PDO::PARAM_INT);
            $withdrawStatement->execute();
        }

        // Commits the acceptance and the withdrawals simultaneously
        $this->conn->commit();
    }
}

```

Figure 5.33: Auto-Withdrawal Implementation After Supervisor Acceptance

3. Secure File Upload Pipeline (Proposal Validation)

Accepting user-uploaded files poses a significant security risk to any web server. To satisfy Functional Requirements 1.2 and 1.3 of the Request & Proposal module, the system implements a strict multi-layered file validation pipeline within the ProposalStorageDAO.

Rather than executing these checks sequentially in the controller, the data access layer encapsulates the entire security protocol, ensuring that no file touches the permanent storage directory unless it passes three distinct gates: payload size, header verification, and extension matching.

Payload Size Constraint

To prevent storage bloat and potential Denial of Service (DoS) attacks via massive file uploads, the DAO explicitly defines a 5.0 MB constraint in bytes (MAX_PROPOSAL_SIZE = 5242880). It evaluates the temporary file size immediately, rejecting oversized payloads before initiating any heavy read operations.

```
private const MAX_PROPOSAL_SIZE = 5242880; // 5MB in bytes
```

Figure 5.34: 5.0 MB constraint in bytes

```
class ProposalStorageDAO {  
    public function storeProposalPDF($uploadedFile, $studentID) {  
        /*  
        |-----  
        | File Size Validation  
        |-----  
        */  
        if ((int) $uploadedFile["size"] > self::MAX_PROPOSAL_SIZE) {  
            return ["success" => false, "message" => self::INVALID_FILE_MESSAGE];  
        }  
    }  
}
```

Figure 5.35: Evaluate Temporary File Size Before Any Read Operations

MIME-Type Header Verification and Storage

A common vulnerability in legacy systems is trusting the `$_FILES['type']` parameter, which is generated by the client browser and easily spoofed by malicious actors. To secure the pipeline, the system utilises PHP's `finfo` (File Information) extension. This forces the server to deeply inspect the physical magic bytes within the file's header to determine its true MIME type.

The system validates that the genuine MIME type evaluates to `application/pdf` and cross-references it against a sanitised lowercase extension string. Only when both checks succeed does the system invoke the native `move_uploaded_file()` method, safely transferring the document from temporary memory into a structured directory (e.g., `storage/proposals/{studentID}_{timestamp}.pdf`) and returning the string locator to be committed to the database.

```
/*
|-----
| MIME Type Validation
|-----
| Ensures only PDF proposal documents are accepted.
*/
// Gate 2: True MIME-type inspection via server-side 'finfo' (FR 1.3)
$finfo = new finfo(FILEINFO_MIME_TYPE);
$mimeType = $finfo->file($uploadedFile["tmp_name"]);

// Gate 3: Extension sanitization
$extension = strtolower(pathinfo($uploadedFile["name"] ?? "", PATHINFO_EXTENSION));

if ($mimeType !== "application/pdf" || $extension !== "pdf") {
    return ["success" => false, "message" => self::INVALID_FILE_MESSAGE];
}
```

Figure 5.36: Evaluate MIME type and cross-references against sanitised lowercase extension string in ProposaStorageDAO

```
/*
|-----
| Move Uploaded Proposal
|-----
*/
// Physical file routing to secure server directory
if (!move_uploaded_file($uploadedFile["tmp_name"], $destination)) {
    return ["success" => false, "message" => "Unable to store proposal document."];
}
```

Figure 5.37: Physical File Routing in ProposalStorageDAO

5.1.4 Client-Side Execution and UI synchronisation

Because PHP operates synchronously and executes only upon receiving an HTTP request, the presentation layer cannot rely solely on the backend for real-time visual updates or immediate user feedback. To bridge this gap, the system utilises vanilla JavaScript to manipulate the Document Object Model (DOM) and intercept browser events, ensuring a responsive User Experience (UX) without requiring constant page reloads.

1. Client-Side DOM Manipulation (Real-Time Visuals)

To satisfy the requirement for a live timeline countdown (FR 2.2 of the Timeline Module), the system must continuously update the interface every second. Rather than querying the database every 1,000 milliseconds, which would instantly crash the server, the JavaScript utilises a pre-calculated `serverOffsetMs` to safely execute the countdown using the client's local browser clock.

The `renderCountdown()` function recalculates the exact remaining seconds mathematically. It then utilises the `textContent` property to physically mutate the HTML DOM node in real-time. This allows the visual timer to tick down seamlessly on the frontend while remaining mathematically synchronised with the backend Singleton engine.

```

document.addEventListener("DOMContentLoaded", function() {
  const timer = document.getElementById("countdownValue");
  if (!timer) return;

  let timelinePayload = window.ssasTimelineData || {};
  let serverOffsetMs = (Number(timelinePayload.serverEpoch || 0) * 1000) - Date.now();

  function pad(value) { return String(value).padStart(2, "0"); }

  function formatRemaining(seconds) {
    seconds = Math.max(0, Math.floor(seconds));
    const days = Math.floor(seconds / 86400);
    const hours = Math.floor((seconds % 86400) / 3600);
    const minutes = Math.floor((seconds % 3600) / 60);
    return `${pad(days)}d ${pad(hours)}h ${pad(minutes)}m`;
  }

  function formatDateTime(value) {
    if (!value) return "-";
    const date = new Date(value.replace(" ", "T"));
    return date.toLocaleString("en-MY", { day: "2-digit", month: "short", year: "numeric", hour: "2-digit", minute: "2-digit" });
  }

  function renderCountdown() {
    const target = timelinePayload.activePhase ? timelinePayload.endTimeStamp : (timelinePayload.startTimeStamp || null);
    let remainingSeconds = 0;
    if (target) {
      // Calculates remaining time by comparing target against local time + server offset
      remainingSeconds = Math.max(0, Math.floor((new Date(target.replace(" ", "T")).getTime() - (Date.now() + serverOffsetMs)) / 1000));
    }
    // Physically mutates the DOM text without reloading the page
    timer.textContent = formatRemaining(remainingSeconds);
  }

  function renderPayload(payload) {
    timelinePayload = payload;
    serverOffsetMs = (Number(payload.serverEpoch || 0) * 1000) - Date.now();
    document.getElementById("serverClock").textContent = "Server Time: " + payload.serverTime;
    document.getElementById("phaseStatus").textContent = payload.phaseStatus;
    document.getElementById("phaseTitle").textContent = payload.activePhaseName;
    document.getElementById("phaseMessage").textContent = payload.message;
    document.getElementById("submissionAccess").textContent = payload.canSubmitProposal ? "Unlocked" : "Locked";
    document.getElementById("phaseStart").textContent = formatDateTime(payload.startTimeStamp);
    document.getElementById("phaseEnd").textContent = formatDateTime(payload.endTimeStamp);
    document.getElementById("countdownCaption").textContent = payload.endTimeStamp ? "Ends " + formatDateTime(payload.endTimeStamp) : "No active deadline";
    renderCountdown();
  }

  // Re-syncs the UI with the authoritative server clock
  async function refreshTimeline() {
    try {
      const response = await fetch("../server/application/student/getTimelineStatus.php", { credentials: "same-origin" });
      if (response.ok) renderPayload(await response.json());
    } catch (error) { /* Silent fail */ }
  }

  // Local 1000ms visual tick
  renderCountdown();
  setInterval(renderCountdown, 1000); // Executes the DOM mutation every 1000 milliseconds
  setInterval(refreshTimeline, 60000); // Authoritative server poll to prevent browser desynchronization
});

```

Figure 5.36: Live Timeline Countdown Implementation

2. Form Event Interception (Client-Side Validation)

While the backend architecture strictly enforces data security (such as the Facade Pattern validation), relying entirely on server-side validation results in a poor user experience, as the user must wait for a full page reload only to receive an error.

To optimise performance and reduce unnecessary server load, the `student.js` script actively intercepts form submissions. By attaching an event listener to the form, the script executes a suite of validation constraints (e.g., checking if an uploaded avatar exceeds the 2.0 MB limit). If a constraint is violated, the script invokes `event.preventDefault()`. This physically halts the browser's native synchronous POST request, immediately returning an error state to the user without ever touching the backend network.


```

document.addEventListener("DOMContentLoaded", function() {
  const profileForm = document.getElementById("studentProfileForm");

  // Safety check: Exit if we are not on the Profile page
  if (!profileForm) return;

  const avatarFile = document.getElementById("avatarFile");
  const avatarPreview = document.getElementById("avatarPreview");
  const avatarFileName = document.getElementById("avatarFileName");
  const bio = document.getElementById("personalBio");
  const bioCounter = document.getElementById("bioCounter");
  const tagCounter = document.getElementById("tagCounter");
  const selectedTags = document.getElementById("selectedTags");
  const tagInputs = Array.from(document.querySelectorAll('input[name="interestTags[]"]'));
  const maxAvatarBytes = 512 * 1024;

  function updateBioCounter() {
    if (bio && bioCounter) {
      bioCounter.textContent = bio.value.length + " / 500";
    }
  }

  function updateTags() {
    if (!tagCounter || !selectedTags) return;

    const selected = tagInputs.filter(input => input.checked);

    tagCounter.textContent = selected.length + " / 5 active";
    selectedTags.innerHTML = "";

    selected.forEach(function(input) {
      const tag = document.createElement("span");
      tag.className = "selected-tag";
      tag.textContent = input.dataset.name;
      selectedTags.appendChild(tag);
    });

    if (selected.length === 0) {
      const empty = document.createElement("span");
      empty.className = "selected-tag";
      empty.textContent = "No interests selected";
      selectedTags.appendChild(empty);
    }

    tagInputs.forEach(function(input) {
      input.closest(".tag-option").classList.toggle("selected", input.checked);
    });

    tagInputs.forEach(function(input) {
      input.addEventListener("change", function() {
        const selected = tagInputs.filter(item => item.checked);

        if (selected.length > 5) {
          input.checked = false;
          alert("You can select a minimum of 1 and a maximum of 5 research interests . Please remove one to add another.");
        }

        updateTags();
      });
    });
  }
});

```

Figure 5.37: Form event Interception (Part 1)

```
if (avatarFile) {
  avatarFile.addEventListener("change", function() {
    const file = avatarFile.files[0];

    if (!file) {
      avatarFileName.textContent = "JPG or PNG, max 2.0MB.";
      return;
    }

    if (!["image/jpeg", "image/png"].includes(file.type) || file.size > maxAvatarBytes) {
      alert("Upload failed. Please ensure your profile picture is in JPG or PNG format and does not exceed 2.0MB.");
      avatarFile.value = "";
      avatarFileName.textContent = "JPG or PNG, max 2.0MB.";
      return;
    }

    avatarFileName.textContent = file.name;
    avatarPreview.innerHTML = '';
  });
}

profileForm.addEventListener("submit", function(event) {
  const selected = tagInputs.filter(input => input.checked);

  if (bio && bio.value.length > 500) {
    event.preventDefault();
    alert("Validation Error - Personal Bio cannot exceed 500 characters.");
    return;
  }

  if (selected.length === 0 || selected.length > 5) {
    event.preventDefault();
    alert("You can select a minimum of 1 and a maximum of 5 research interests . Please remove one to add another.");
    return;
  }
});

profileForm.addEventListener("reset", function() {
  setTimeout(function() {
    updateBioCounter();
    updateTags();
    if (avatarFileName) avatarFileName.textContent = "JPG or PNG, max 2.0MB.";
  }, 0);
});

updateBioCounter();
updateTags();
if (bio) {
  bio.addEventListener("input", updateBioCounter);
}
});
```

Figure 5.38: Form event Interception (Part 2)

5.2 Testing

The software testing phase is a critical component of the Software Development Life Cycle (SDLC) designed to systematically evaluate the Supervisor Selection and Allocation System (SSAS). The primary purpose of this phase is to rigorously verify that the developed application accurately satisfies the Functional Requirement and baseline **Non-Functional Requirements (NFR)** established during the system design phase. By executing targeted test cases, this phase aims to identify structural defects, validate the integrity of the backend business logic (such as concurrency limits and temporal phase locks), and ensure the overall system reliability.

The testing scope comprehensively covers the core Student-Based Modules: Availability & Status, Discovery & Search, Request & Proposal, Student Profile, Student Review, and Timeline & Milestone. In addition to functional evaluation, **targeted non-functional attributes, specifically Role-Based Access Control (RBAC) security and strict file capacity constraints (e.g., the 5.0 MB proposal upload limit), will be tested to ensure the system is secure and resilient against payload vulnerabilities.**

To achieve high accuracy and efficiency during the testing execution, Katalon Studio is employed as the primary automated testing framework. **While targeted non-functional boundary tests are conducted manually,** Katalon Studio automates the execution of complex functional test cases, such as multi-criteria filtering and UI state transitions. Utilising an automated tool eliminates human error during repetitive test cycles, provides standardised execution logs for the summary report, and enables rapid regression testing to guarantee that new code modifications do not break existing system functionalities.

5.2.1 Test Items

The test items refer to the modules and functions that will be tested in this project. The testing focuses on six main student-centric modules: Supervisor Availability & Status Module, Discovery & Search Module, Request & Proposal Module, Student Profile Module, Student Review Module, and Timeline & Milestone Module.

Firstly, the **Supervisor Availability & Status Module** includes the Real-Time Dashboard, Dynamic Status Badges, and Submission Blocking. The Real-Time Dashboard retrieves the supervisor's active supervisee count and predefined administrative role limit to mathematically calculate the real-time quota. The Dynamic Status Badges assign a status of "Available" or "Full" based on these limits and render the corresponding visual badge. The Submission Blocking function evaluates this state to physically disable the "Apply" button and block any backend POST requests if a supervisor is in the "Full" state. Testing this module ensures that capacity calculations are mathematically accurate, visual badges render the correct state, and students are securely prevented from over-booking fully occupied supervisors.

Secondly, the **Discovery & Search Module** includes the Multi-Criteria Filtering Engine and the Recommendation Algorithm. The Multi-Criteria Filtering Engine provides interactive elements for students to input filtering parameters for Academic Programme, Research Interest, and Availability Status, subsequently executing a multi-conditional database query to refresh the interface. The Recommendation Algorithm algorithmically compares the student's saved "Interest Tags" array against the "Expertise Tags" of active supervisors to calculate a numerical matching score and extract the top three highest-scoring supervisors. Testing this module ensures that database queries execute accurately based on user inputs, overlapping tag logic functions correctly, and matched supervisors who are "Full" are mathematically excluded from the results.

Thirdly, the **Request & Proposal Module** includes Digital Submission and Status Tracking. The Digital Submission function handles the upload of the project proposals, enforcing a strict 5.0 Megabytes (MB) file size constraint and validating the application/pdf MIME type before securely storing the document. The Status Tracking function assigns a default "Pending" status, generates a 72-hour Time-to-Live (TTL0 timestamp, and executes a conflict-resolution script to automatically mutate remaining proposals to "Withdrawn" once a student receives an "Accepted" status. Testing this module ensures that file payloads are securely validated, concurrency limits (maximum of three active pending proposals) are strictly enforced, and automated state transitions execute perfectly without generating orphaned data.

Fourthly, the **Student Profile Module** includes Personal Details and Interest Tagging. The Personal Details function allows students to update mutable profile data (such as Personal Bio and External URLs) while strictly locking immutable identification attributes like Student ID, Programme and CGPA. The Interest Tagging function allows the student to select a minimum of one and a maximum of five distinct research interest tags. Testing this module ensures that immutable fields cannot be maliciously altered, character constraints (500-character bio limit) and Regular Expression (Regex) URL validations are enforced, and avatar file payloads strictly adhere to the 2.0 MB limit.

Fifthly, the **Student Review Module** includes the Structured Review System and the Anonymous Review Toggle. The Structured Review System provides an evaluation interface allowing students to input a quantitative rating (1 to 5 stars) and qualitative text feedback, provided they possess a valid supervisor allocation record. The Anonymous Review Toggle parses a boolean state to render the public author string as “Anonymous Student” on the UI while permanently binding the true Student ID within the backend database. Testing this module ensures that reviews are restricted only to valid allocated students, quantitative ratings are aggregated correctly, and identity masking is correctly applied without compromising backend administrative traceability.

Lastly, the **Timeline & Milestone Module** includes the Phase Tracker and Deadline Countdown. The Phase Tracker compares the centralised server time against retrieved database timestamps to mathematically evaluate the active operational phase. The Deadline Countdown calculates the remaining time allowance, updates the visual timer dynamically, and automatically invokes a global system lock to instantly revoke submission rights when the countdown reaches zero. Testing this module ensures that temporal boundary comparisons are calculated correctly, the UI visual timer ticks continuously, and system-wide submission locks trigger perfectly the exact moment a phase deadline expires.

In addition to the core functional modules, testing covers targeted **Non-Functional Requirements (NFR)**. This specifically focuses on Safety and Security Requirements to ensure Role-Based Access Control (RBAC) prevents student accounts from accessing administrative interfaces, and Capacity Requirements to physically verify that the system securely rejects digital file uploads exceeding the defined 5.0 MB payload limit.

5.2.2 Test Scope

The test scope defines the boundaries of the testing phase for the Supervisor Selection and Allocation System (SSAS), detailing the specific types of testing executed to validate the system's reliability and accuracy. The primary focus of this phase is the comprehensive evaluation of the six student-centric modules: the Supervisor Availability & Status Module, Discovery & Search Module, Request & Proposal Module, Student Profile Module, Student Review Module, and Timeline & Milestone Module.

To guarantee system stability prior to deployment, the testing scope is divided into the following distinct categories:

- **Functional Testing:** This forms the core of the testing phase. It evaluates the backend business logic and algorithms to ensure they strictly execute the defined Functional Requirements (FRs). This includes verifying the mathematical accuracy of the Multi-Criteria Filtering Engine, the Recommendation Algorithms' tag-matching logic, the calculation of real-time capacity limits, and the automated execution of state transitions (such as auto-withdrawing pending requests once a proposal is accepted).
- **User Interface (UI) Testing:** This ensures that the frontend presentation layer accurately reflects the backend database state and responds correctly to user interactions. Testing covers the dynamic rendering of visual elements, including the real-time updates of the "Available" and "Full" Status Badges, the accurate calculation and visual ticking of the Timeline Countdown timer, and the correct rendering of the aggregated Supervisor Review scores.
- **Input Validation Testing:** This verifies that the system securely handles user-supplied data before interacting with the database. The scope includes testing character constraints (e.g., the 500-character limit on personal bios), evaluating Regular Expression (Regex) filters on external URLs, ensuring the 1-to-5 star rating limits are respected, and rigorously verifying the physical payload pipelines (e.g., enforcing MIME-type and extension restrictions for .pdf, .jpg, and .png upload)
- **Error Handling and Negative Testing:** This focuses on testing the system's resilience by deliberately simulating invalid actions and constraints violations. The goal is to prove that the system safely intercepts these actions without crashing. Key negative tests include attempting to bypass the Submission blocking logic for a "Full" supervisor, trying to submit a proposal after the "Submission Phase" has expired, uploading oversized files, and attempting to submit duplicate supervisor reviews.

Non-Functional Testing Scope Boundaries

While comprehensive Functional Testing forms the primary scope, a strictly targeted subset of Non-Functional Requirements (NFRs) will also be tested to ensure the system meets baseline operational and security standards. Specifically, testing will manually verify Role-Based Access Control (RBAC) security (ensuring students cannot navigate to administrative dashboards) and strict Capacity constraints (verifying the system actively rejects file uploads exceeding 5.0 MB).

Conversely, heavy stress testing, such as simulating 500 concurrent student users during peak hours, and infrastructure-level validations (such as HTTPS certificate provisioning and 150 GB server storage testing) fall outside the scope of this local development phase. These specific NFRs are recommended for future, production-level deployment testing.

5.2.3 Risk Register

The Risk Register identifies potential hazards and bottlenecks that could negatively impact the execution, accuracy, or timeline of the software testing phase. By proactively defining these risks, their potential impact levels, justification, and corresponding mitigation strategies, the testing process can proceed systematically without causing significant delays to the overall project schedule.

Table 5.1 outlines the primary testing risks associated with the Supervisor Selection and Allocation System (SSAS), specifically focusing on the challenges of testing automated algorithms, temporal deadlines, and local environment constraints.

Table 5.1: Software Testing Risk Register

Risk ID	Risk	Description	Impact & Justification	Mitigation Strategies
TR-01	Automated Script Brittleness (Katalon Studio)	Automated UI tests executing in Katalon Studio may fail or yield false negatives if web interface elements load asynchronously or if dynamic DOM IDs change.	High: False negatives would halt the automated testing pipeline, causing severe delays and required tedious manual re-verification of already functional code.	Utilise relative and highly stable XPath / CSS selectors for element identification. Implement explicit “wait-for-element” commands in Katalon rather than hardcoded time delays to ensure the UI is fully loaded.
TR-02	Temporal Testing Delays	Testing the Timeline & Milestone module and the 72-hour Time-to-Live (TTL) auto-rejection script in real-time is logically impossible within the testing schedule	High: Waiting 72 hours to validate a single test case would entirely stall the testing phase, making it impossible to complete the project within the academic deadline	Temporarily modify the backend test environment to accept a mocked currentTime variable, or manually manipulate the database phase timestamps to simulate the passing of days/hours in seconds.
TR-03	Insufficient Test Data for Algorithms	The Recommendation Algorithm and Multi-Criteria Filtering Engine require a populated database to evaluate sorting accuracy. Testing with only 2 or 3 records will fail to validate the $O(n)$ ranking logic.	Medium: Without a sufficiently complex dataset, the algorithms’s overlapping tag logic cannot be mathematically proven, potentially masking matching errors.	Generate and inject a comprehensive mock dataset (e.g., 20+ dummy supervisor profiles with heavily overlapping and conflicting “Expertise Tags”) directly into the database prior to test execution.

TR-04	Concurrency Simulation Limitations	The local XAMPP testing environment may struggle to simulate true high-volume concurrency (e.g., multiple students clicking “Apply” at the exact same millisecond”) to verify ACID row-level locking.	Medium: While the core Submission Blocking logic can be validated sequentially, the inability to simulate a true server stress-test leaves a major minor theoretical risk of race conditions unproven.	Utilise Katalon Studio’s data-driven testing features to execute rapid, sequential POST requests, and manually inspect the database state to ensure the constraint correctly rejects the overflow applications.
TR-05	File System Permission Errors	The Secure File Upload tests (verifying the 5.0 MB PDF limit) may fail due to local OS read/write permission issues in the server directory, rather than a failure of the PHP logic.	Low: This is a local environment configuration issue rather than a structural software bug. It can typically be diagnosed and resolved in minutes.	Verify and configure explicit Read/Write permissions for the local Apache/XAMPP server directory before commencing the Request & Proposal module testing phase.

5.2.4 Testing Strategy and Technique

The testing strategy defines the methodological approach used to validate the SSAS. Given that the primary goal of this phase is to ensure the system satisfies the user-centric Functional Requirements (FRs) defined in Chapter 3, the testing strategy is strictly rooted in Black Box Testing.

Justification for Black Box Testing

Black Box Testing evaluates the software from the end-user's perspective without requiring knowledge of the internal backend source code. This approach is highly appropriate for the SSAS because the system is driven by strict administrative business rules (e.g., quota limits, timeline phases, and eligibility). Black Box testing ensures that when specific inputs are provided (e.g., submitting a proposal), the system yields the exact expected output (e.g., a "Pending" status or a rejection error) regardless of the underlying PHP/SQL architecture.

To systematically derive test cases for Black Box execution, two formal test design techniques are employed: Decision Tables and Cause-Effect Graphing.

Justification for Decision Tables and Cause-Effect Graphs:

- **Cause-Effect Graphing:** This technique visually maps the logical relationships (AND, OR, NOT) between various system inputs (Causes) and expected outcomes (Effects). It is utilised because it efficiently identifies impossible combinations of inputs, reducing the number of redundant test cases before they are written.
- **Decision Table Testing:** Once the boolean logic is mapped via the Cause-Effect graph, it is translated into a Decision Table. This technique is highly effective for testing complex, multi-conditional business rules. Because SSAS heavily relies on overlapping constraints (e.g., a student can only apply IF the supervisor is available AND the timeline is open AND the student has fewer than 3 pending requests), Decision Tables mathematically guarantee that every possible permutation of these rules is tested.

Additionally, **Boundary Value Analysis (BVA)** is utilised specifically for the Non-Functional Requirements (NFRs) to test exact threshold limits, such as verifying the system rejects a file sized at 5.1 MB while accepting a file sized at exactly 5.0 MB.

Module 1: Supervisor Availability & Status Module**Function 1.1: Real-Time Dashboard**

This function evaluates the retrieval of quota data, the mathematical formatting of the fractional string, and its ability to recalculate upon a database trigger.

Cause-Effect Graph:

Table 5.2: Causes (Real-Time Dashboard)

Causes	Description
C1	Supervisor profile loaded with valid active supervisee count and role limit
C2	A new student proposal for this supervisor is successfully accepted (database commit successful)

Table 5.3: Effects (Real-Time Dashboard)

Effects	Description
E1	Calculate and render the initial quota as fractions string (e.g., “3/8 slots taken”)
E2	Dynamically recalculate and refresh the UI string (e.g., updating to “4/8 slots taken”)
E3	Abort rendering and display an error/fallback state

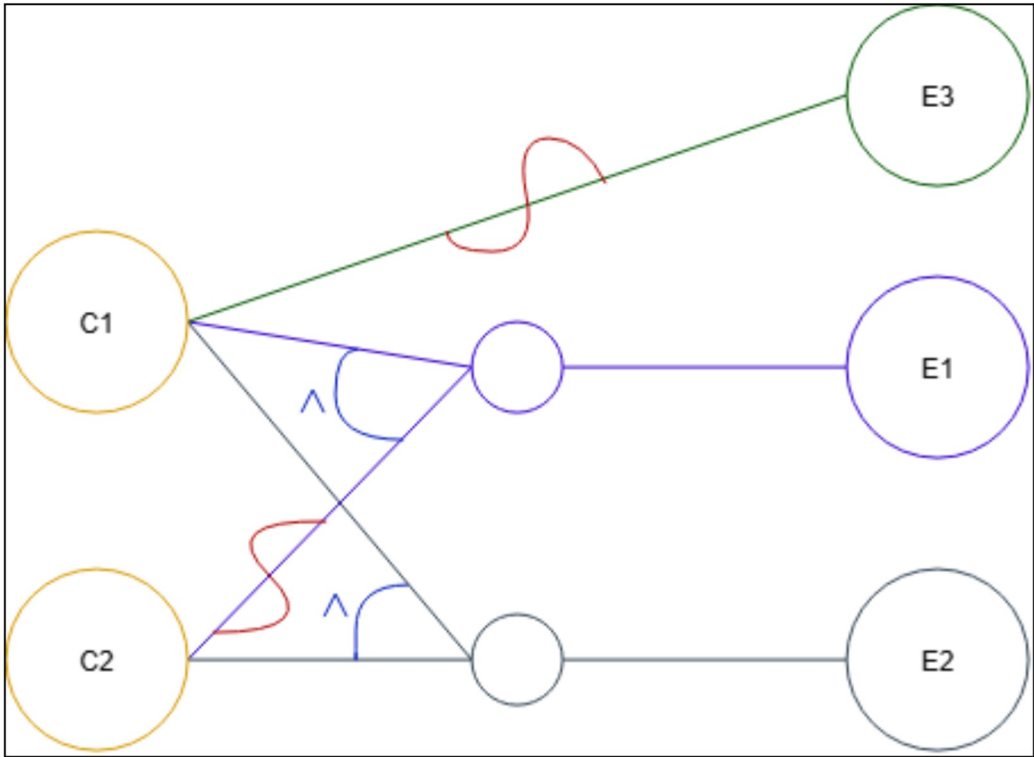


Figure 5.39: Cause-Effect Graph (Real-Time Dashboard)

Decision Table:

Table 5.4: Decision Table (Real-Time Dashboard)

Conditions	TC1	TC2	TC3
C1: Profile loaded with valid quota data	T	T	F
C2: New accepted proposal committed	F	T	-
Actions (Expected Results)			
E1: Render initial fractional string (X/Y)	T		
E2: Recalculate & refresh string (X+I/Y)		T	
E3: Abort rendering / Fallback state			T

In table 5.4, TC1 and TC2 are positive test cases, while TC3 is a negative test case.

Function 1.2: Dynamic Status Badges

This function evaluates the mathematical capacity of a supervisor to assign and render the correct visual badge state.

Cause-Effect Graph:

Table 5.5: Causes (Dynamic Status Badges)

Causes	Description
C1	Active Supervisee Count < Predefined Role Limit

Table 5.6: Effects (Dynamic Status Badges)

Effects	Description
E1	Assign “Available” status & Render Green Badge
E2	Assign “Full” status & Render Red Badge

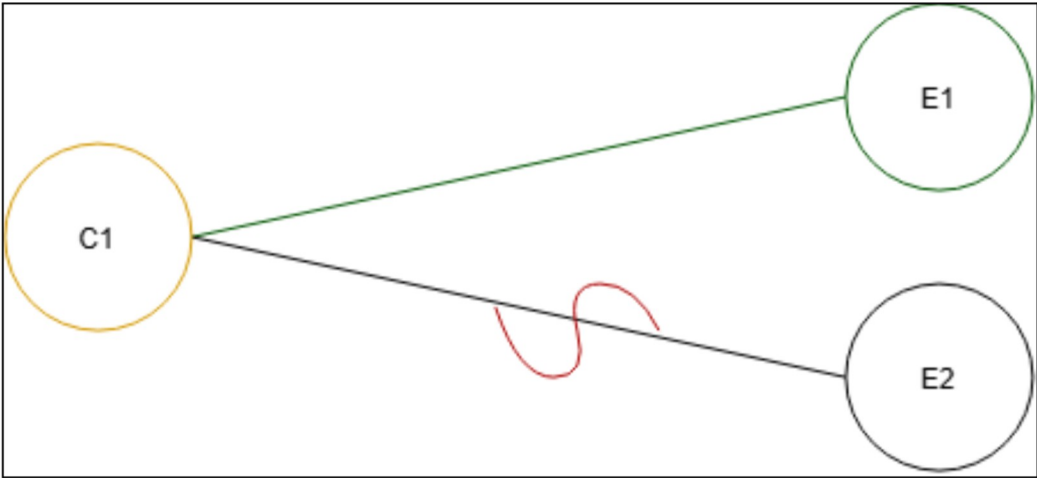


Figure 5.40: Cause-Effect Graph (Dynamic Status Badges)

Decision Table:

Table 5.7: Decision Table (Dynamic Status Badges)

Conditions	TC1	TC2
C1: Active Count < Role Limit	T	F
Actions (Expected Results)		
E1: Render Green “Available” Badge	T	
E2: Render Red “Full” Badge		T

In table 5.7, TC1 is a positive test case while TC2 is a negative test case.

Function 1.3: Submission Blocking

This function acts as the system gatekeeper, physically blocking the application pipeline based on cross-module dependencies and quota constraints.

Cause-Effect Graph:

Table 5.8: Causes (Submission Blocking)

Causes	Description
C1	Target Supervisor is in “Available” Status State
C2	Student has < 3 Active Pending Requests
C3	Active System Phase == “Submission Phase”

Table 5.9: Effects (Submission Blocking)

Effects	Description
E1	Enable “Apply” Button & Allow backend POST Request
E2	Disable “Apply Button, remove hyperlink & Block backend POST Request

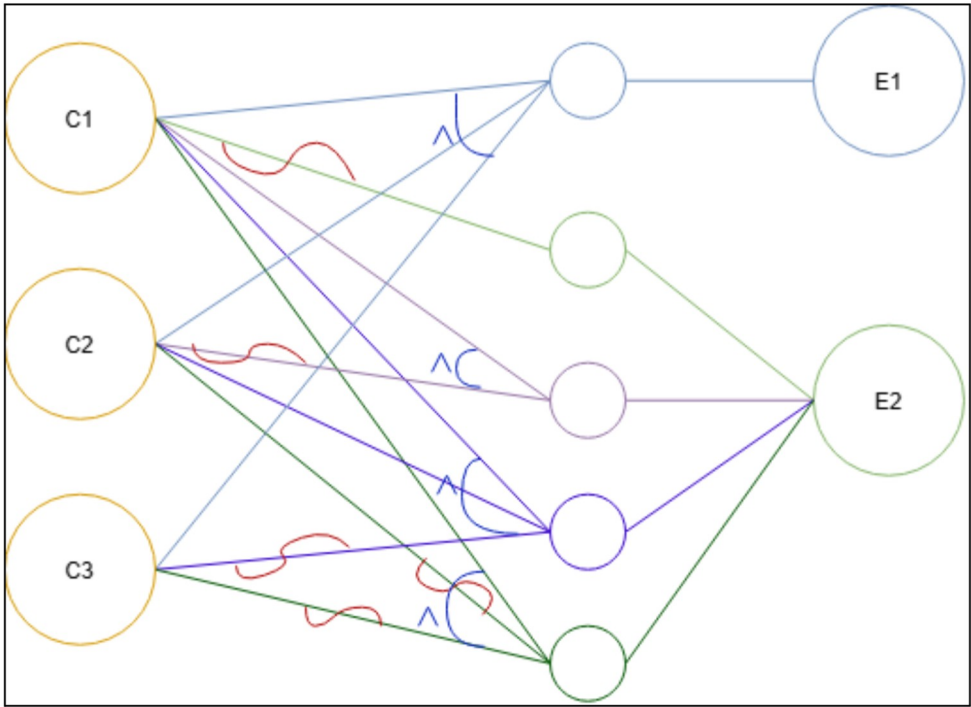


Figure 5.40: Cause-Effect Graph (Submission Blocking)

Decision Table:

Table 5.10: Decision Table (Submission Blocking)

Conditions	TC1	TC2	TC3	TC4	TC5
C1: Supervisor is “Available”	T	F	T	T	T
C2: Student Pending Count < 3	T	-	F	T	F
C3: Phase == “Submission”	T	-	-	F	F
Actions (Expected Results)					
E1: Enable Apply & Allow POST	T				
E2: Disable Apply & Block POST		T	T	T	T

In table 5.10, TC1 is a positive test case, while TC2, TC3, TC4 and TC5 are negative test cases.

Module 2: Discovery & Search Module**Function 2.1: Multi-Criteria Filtering Engine**

This function evaluates each supervisor record sequentially against the student's text and dropdown inputs.

Cause-Effect Graph:

Table 5.11: Causes (Multi-Criteria Filtering Engine)

Causes	Description
C1	Supervisor's name matches the text inputted into the "Search Bar"
C2	Supervisor's programme matches the "Programme" dropdown selection
C3	Supervisor's expertise intersects with the "Research Interest" dropdown selection
C4	Supervisor's actual capacity state matches the "Availability Status" tab selection

Table 5.12: Effects (Multi-Criteria Filtering Engine)

Effects	Description
E1	Retain record and display Supervisor Profile Card on the UI
E2	Remove record from the UI view